

Operating System

HotSpot VM

2025.09.04

CS STUDY

SSAFY

송현우

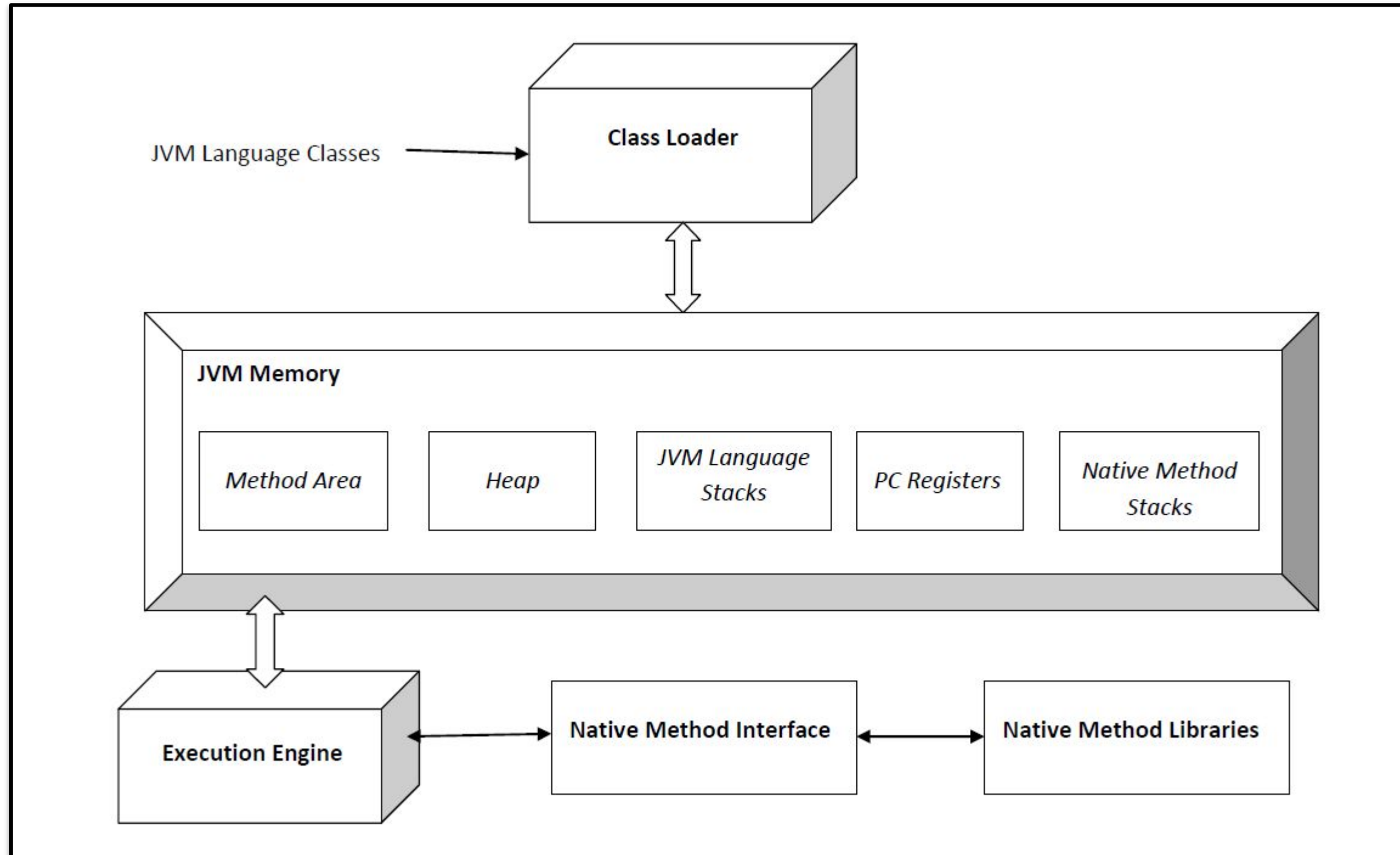
Index

1	JVM	04
2	Class Loader	10
3	Runtime Data Area	22
4	GC	33
5	JIT Compiler	39

JVM

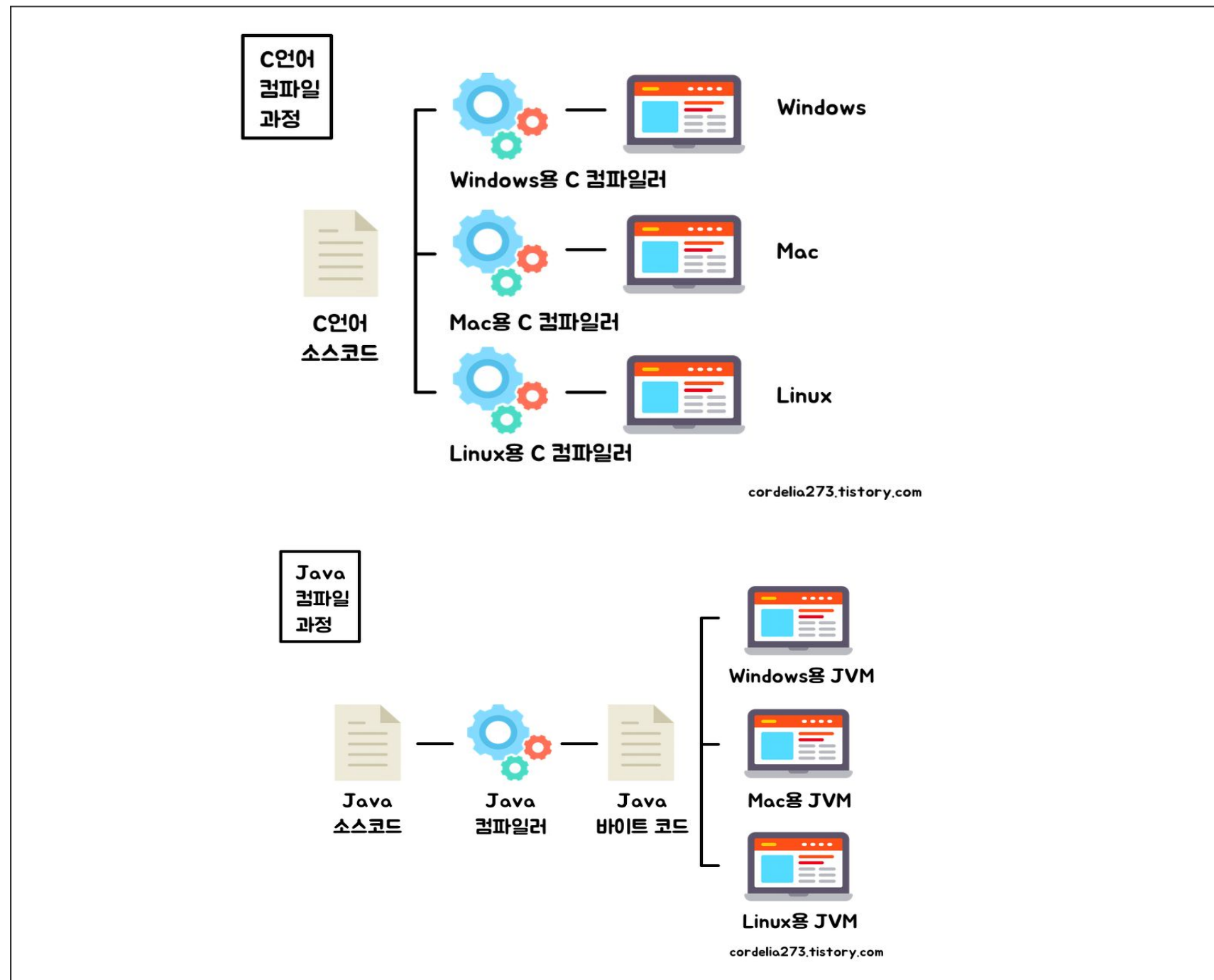
JVM Structure

JVM



Cross-platform software

JVM



WOCA

- Write Once, Compile Anywhere
- C, C++

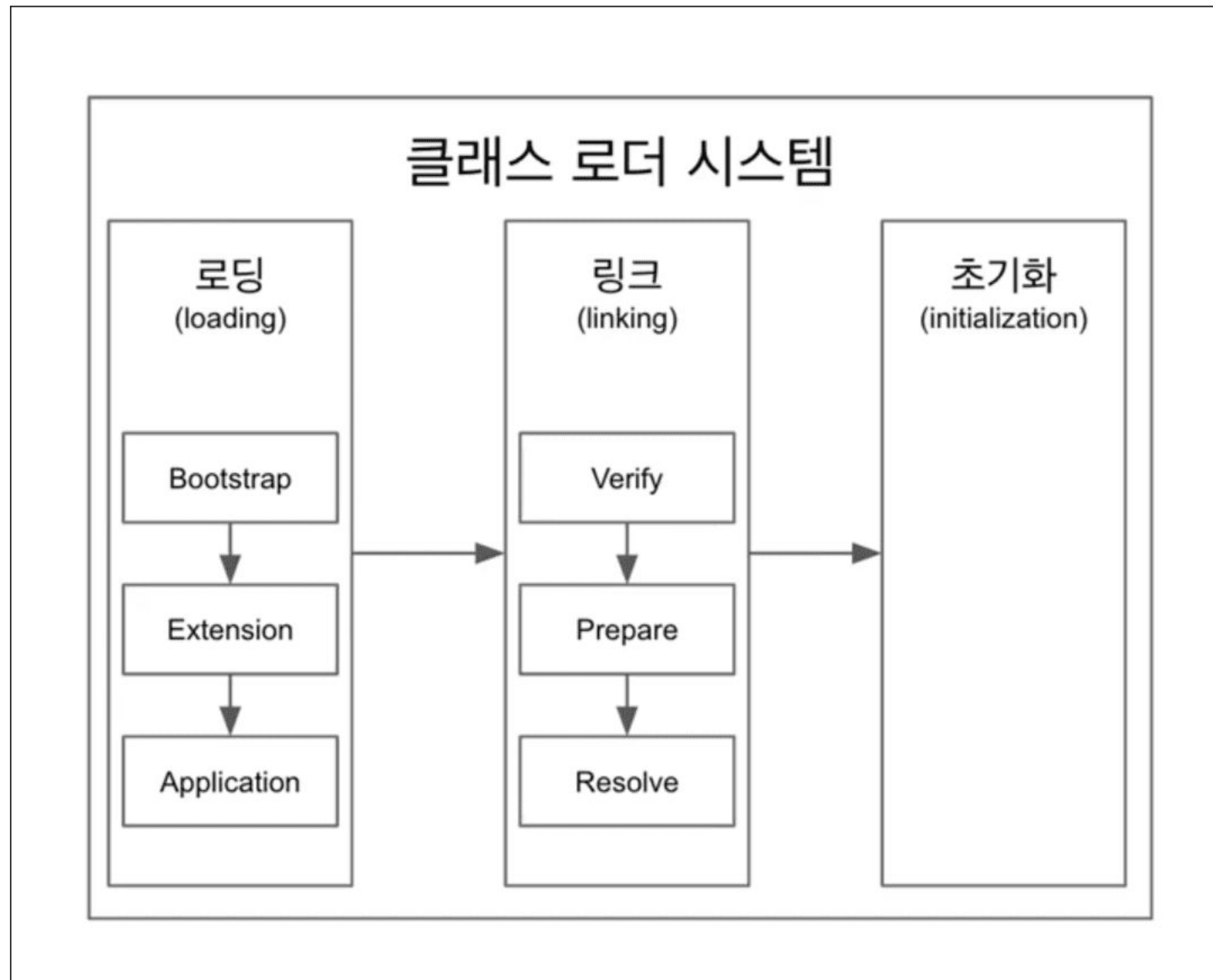
WORA

- Write Once Run Anywhere
- 운영체제에 독립적이다 (JVM 자체는 종속적)
- JVM 언어는 다양하다
 - Java, Kotlin, Groovy...

Class Loader

Class Loader Structure

index



Class Loader

- Class Loader는 런타임 데이터 영역을 채우는 과정을 수행한다.
- 로딩, 링크, 초기화의 세 단계로 이루어진다.
- 계층적(Hierarchical)이다.

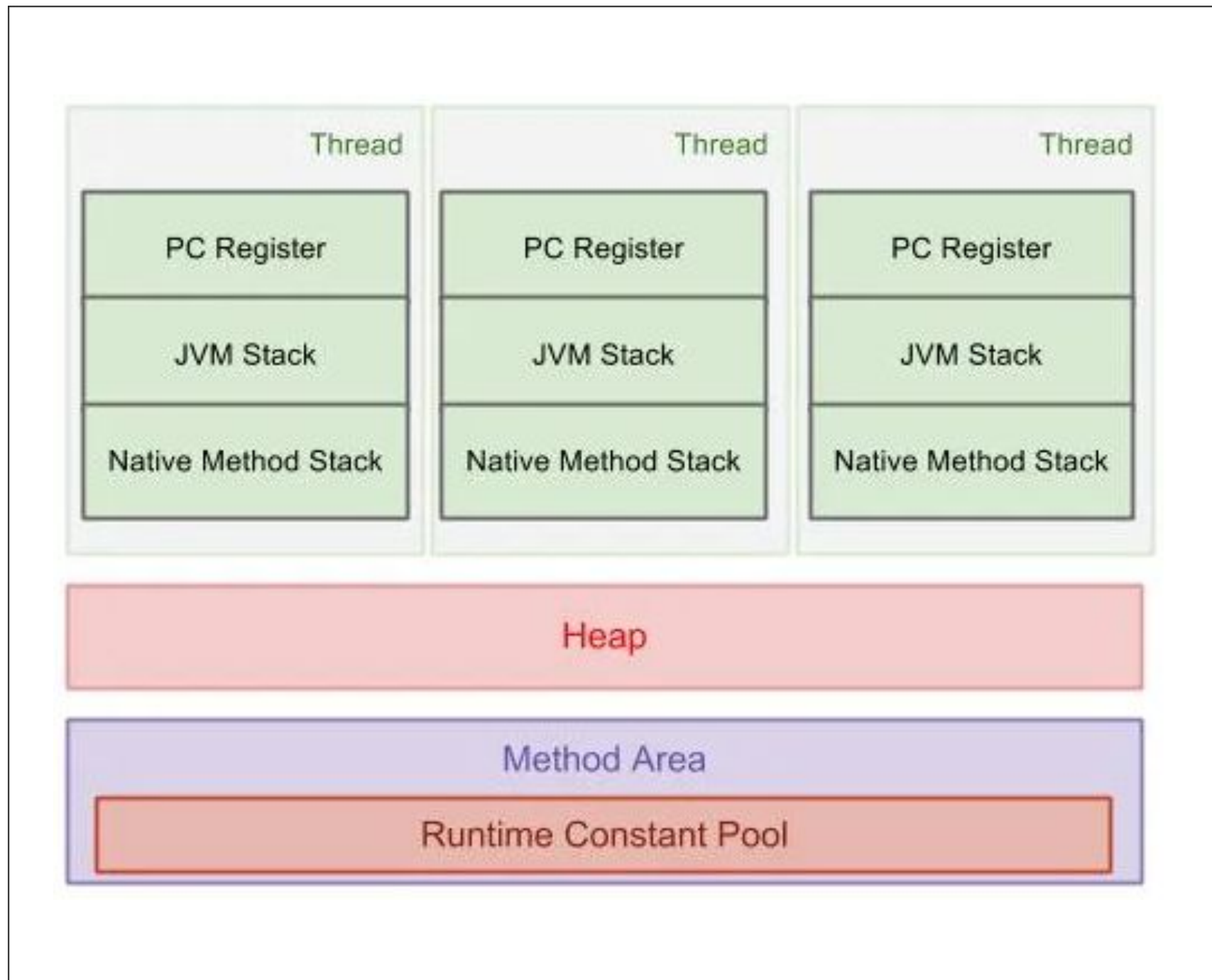
단계 별 핵심

- 로딩 : Class 객체를 생성하여 Heap 영역에 저장
- 링크 : Symbol Resolution & Relocation
- 초기화 : static 변수 할당

Runtime Data Area

Runtime Data Area Structure

Runtime Data Area



Per JVM

- Method Area, Heap Area
- 모든 스레드가 공유하는 영역

Per Thread

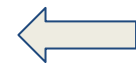
- PC Register, Stack Area, Native Method Area
- 스레드 별로 생성되는 영역

Java ByteCode

Runtime Data Area

```
Compiled from "HelloWorld.java"
public class HelloWorld {
    public HelloWorld();
        Code:
            0: aload_0
            1: invokespecial #1           //
            4: return

    public static void main(java.lang.String[]);
        Code:
            0: getstatic     #7           //
            3: ldc           #13          //
            5: invokevirtual #15          //
            8: return
}
```



```
public class HelloWorld 0개의 사용위치
{
    public static void main(String[] args)
    {
        System.out.println("Hello World!");
    }
}
```

- 메서드 별로 0-based opcode offset을 갖는다
- `aload_0` : 현재 객체(this)를 operand stack에 올린다
- `invokevirtual` : 메서드 호출(dynamic dispatch)

Constant Pool

Runtime Data Area

```
Constant pool:
  #1 = Methodref      #2.#3      // java/la
  #2 = Class          #4          // java/la
  #3 = NameAndType    #5:#6      // "<init>"
  #4 = Utf8           java/lang/Object
  #5 = Utf8           <init>
  #6 = Utf8           ()V
  #7 = Fieldref       #8.#9      // java/la
  #8 = Class          #10         // java/la
  #9 = NameAndType    #11:#12    // out:Lja
#10 = Utf8           java/lang/System
#11 = Utf8           out
#12 = Utf8          Ljava/io/PrintStream;
#13 = String         #14         // Hello W
#14 = Utf8           Hello World!
#15 = Methodref      #16.#17    // java/io
#16 = Class          #18         // java/io
#17 = NameAndType    #19:#20    // println
#18 = Utf8           java/io/PrintStream
#19 = Utf8           println
#20 = Utf8           (Ljava/lang/String;)V
#21 = Class          #22         // HelloWo
#22 = Utf8           HelloWorld
#23 = Utf8           Code
#24 = Utf8          LineNumberTable
#25 = Utf8           main
#26 = Utf8           ([Ljava/lang/String;)V
#27 = Utf8           SourceFile
#28 = Utf8           HelloWorld.java
```

Constant Pool

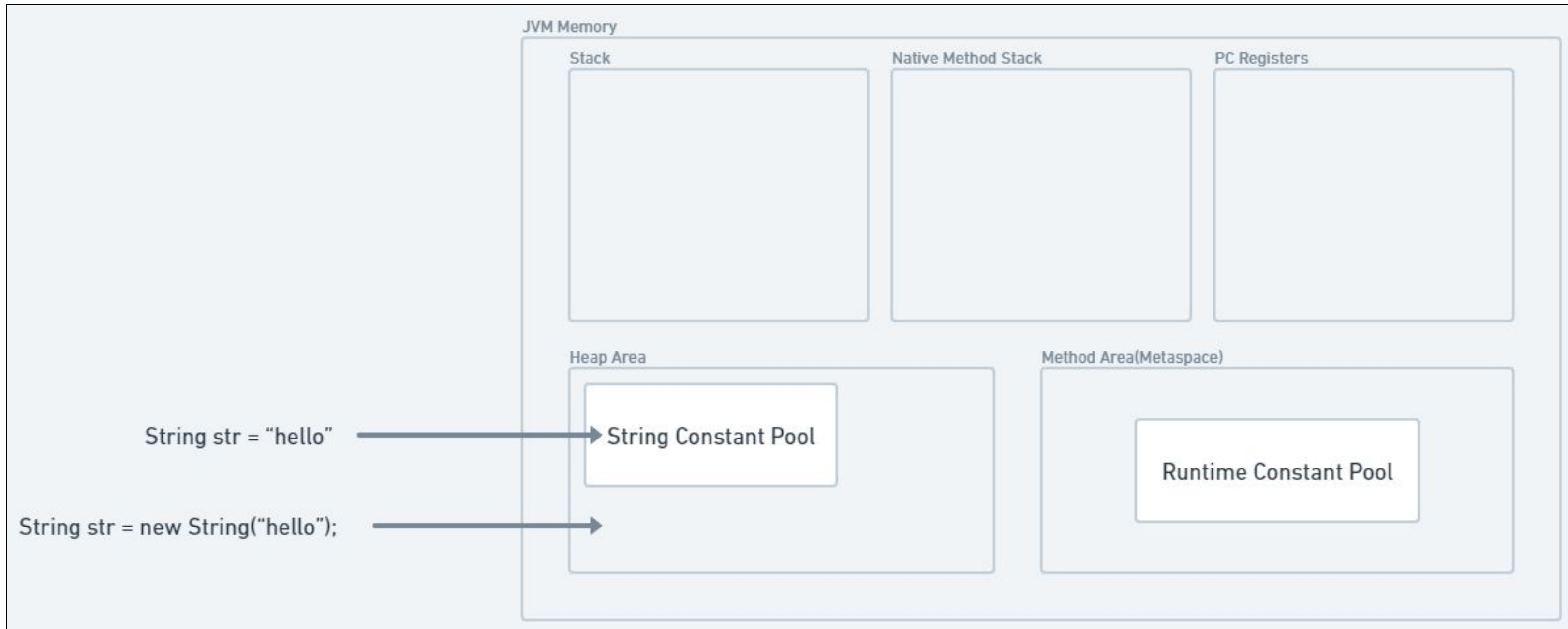
- 컴파일 단계의 심볼릭 참조(symbolic reference)
- #0은 dummy entry

Runtime Constant Pool

- 클래스 로더의 링킹 -> Resolve 과정에서 Symbolic reference를 Direct reference로 변환
- JVM에서는 모든 클래스를 프로그램 실행 시 한 번에 변환하지 않고, 해당 참조가 실제로 필요할 때 **Lazy Resolution**함
- Dynamic Loading
- 런타임 상수 풀이 넘치면 OOM(OutOfMemory)이 발생할 수 있다
 - **java 8부터 Metaspace(native memory 영역)에 위치하면서 개선됨**

String Constant Pool

Runtime Data Area



- Constant Pool ≠ Runtime Constant Pool ≠ String Constant Pool
- 참고 및 출처 : <https://deveric.tistory.com/123>

GC

Keyword

GC

- STW(Stop-the-world)

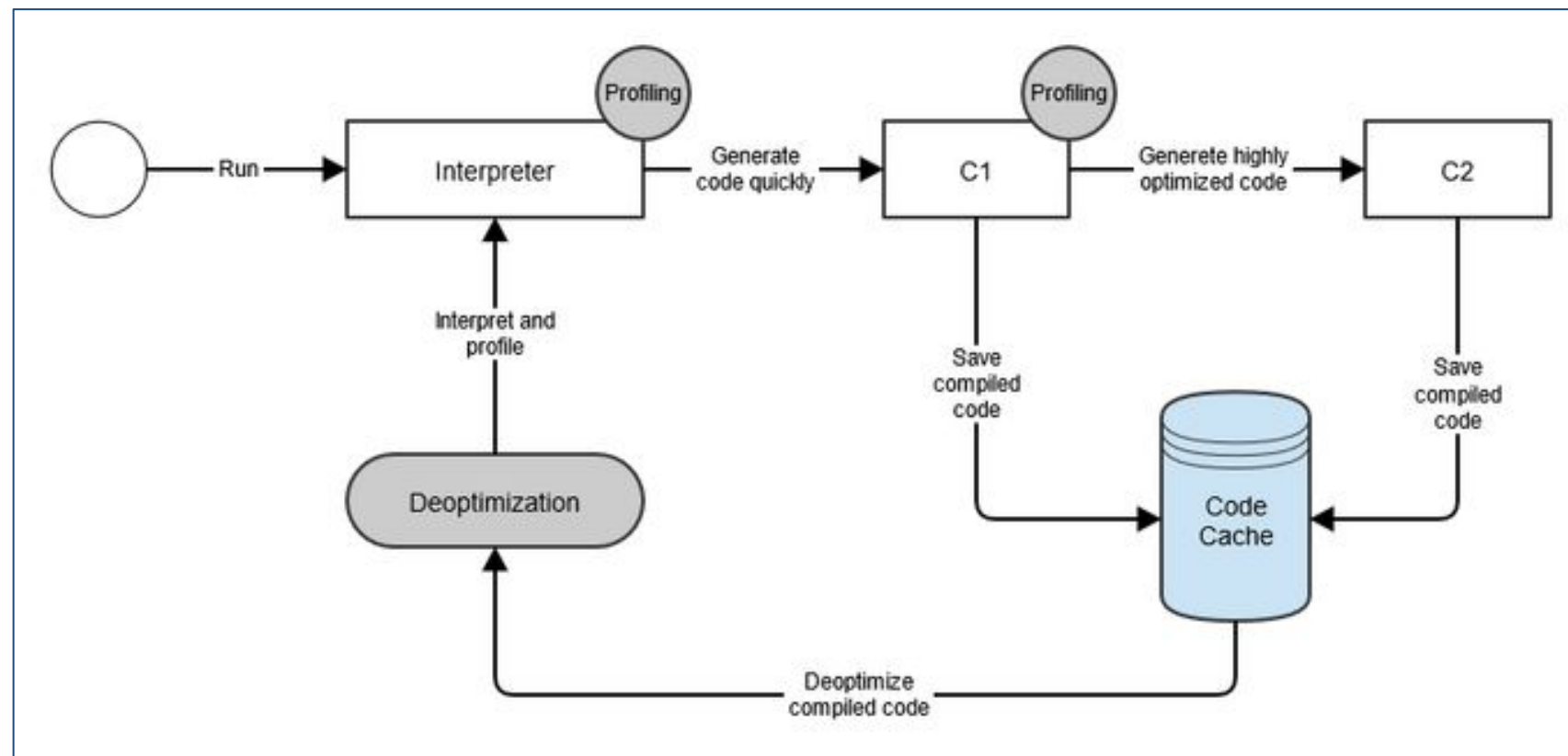
- Memory fragmentation & Compaction

- Generational GC, G1 GC, ZGC

JIT Compiler

Tiered Compilation

JIT Compiler



출처 : <https://hudi.blog/jvm-warm-up/>

Compile Threshold

- InvocationThreshold: 메서드 호출 수
- BackEdgeThreshold: 메서드 내 반복문 횟수
- 최적화 level을 조절하는 기준 중 일부

Deoptimization

- 타입 추측 실패, 다형성, 예외 등 JVM이 세운 가정이 깨지면 deoptimization 발생
- 스레드가 safepoints에 도달할 때까지는 기다림
- stack frame 재구성 & OSR

참고 : <https://medium.com/@AlexanderObregon/what-triggers-jvm-deoptimization-during-runtime-6c61170e4295>

JVM warm-up

JIT Compiler

```
C:\Users\SSAFY\.jdk\openjdk-24.0.2+12-54
--- 워업 전 실행 ---
time: 50517500

--- 워업 중 (5000회 반복) ---

--- 워업 후 실행 ---
time: 15500

--- 추가 워업 중 (1000000회 반복) ---

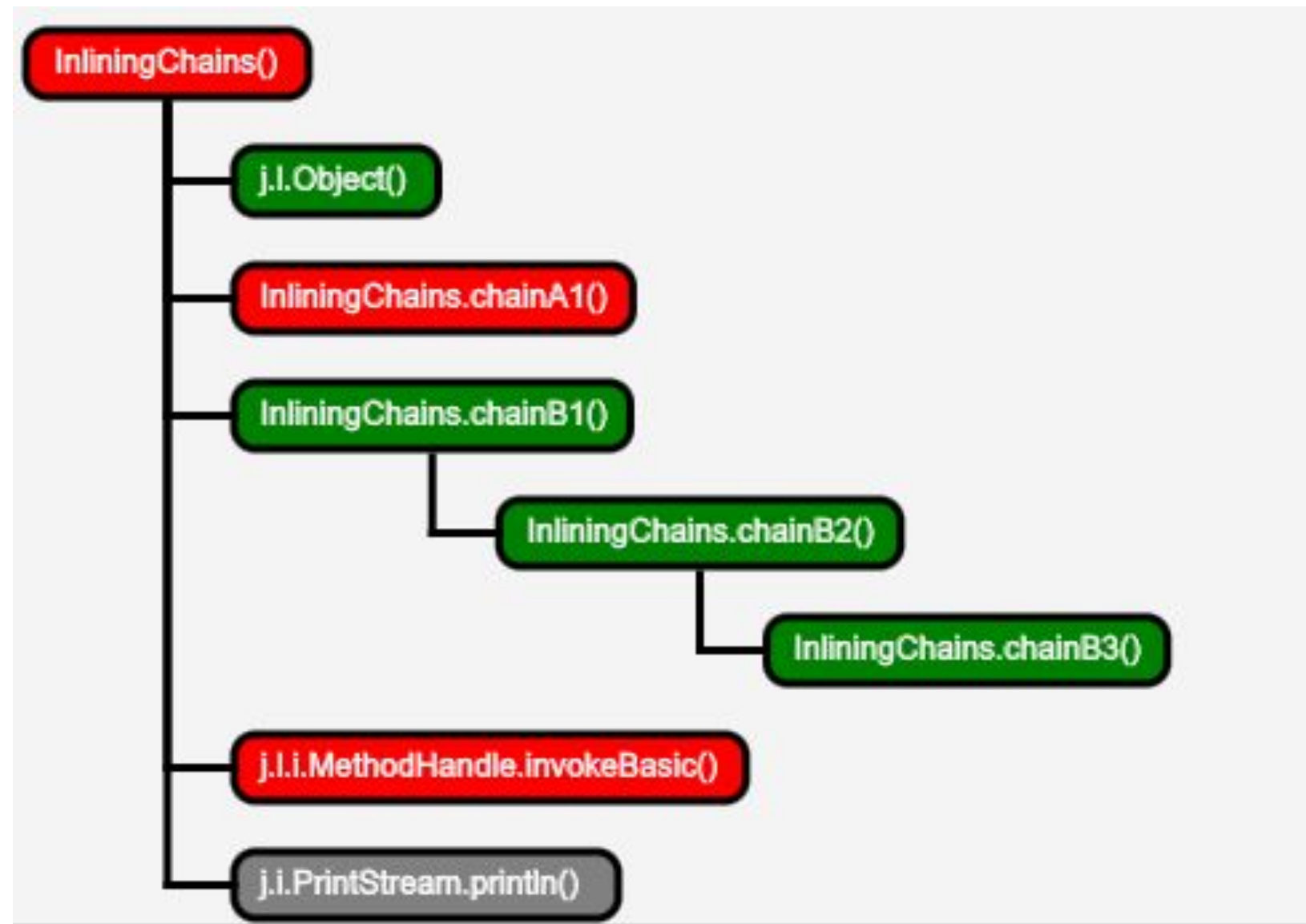
--- 최종 실행 ---
time: 2200
```

JVM warm-up

- Tiered Compilation에 따라 점진적으로 최적화가 이루어진다.
- 따라서, Java Application의 성능을 측정할 때는 warm-up이 충분히 이루어진 후에 해야한다.
- 서버를 처음 측정할 때 발생하는 초기 latency(=cold start)가 심한 이유 중 하나
- 특히, MSA와 서버리스 환경에서 Java를 구동한다면?
 - GraalVM(AOT compiler)

JIT Optimisation

JIT Compiler



OSR(On-Stack-Replacement)

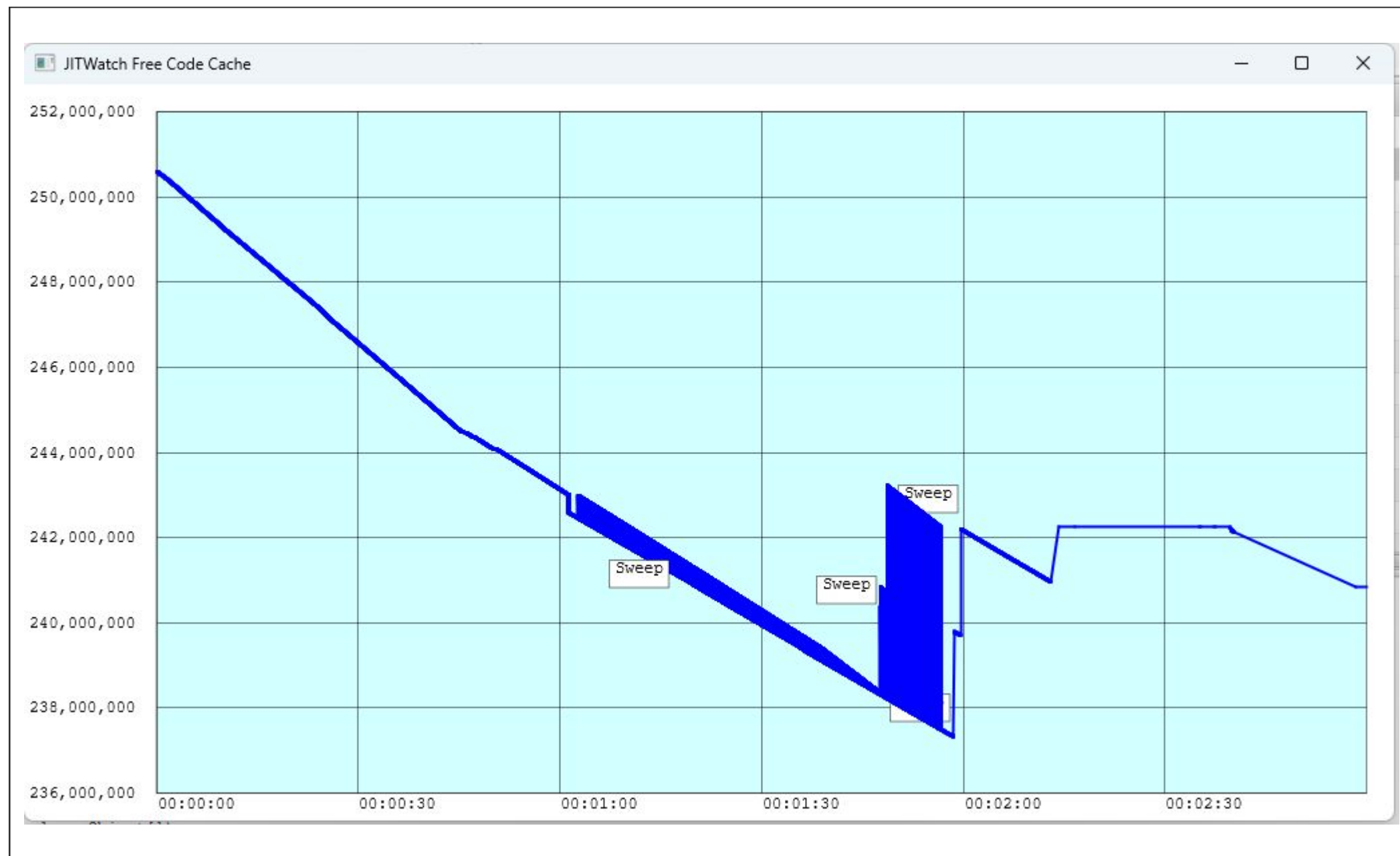
- 장시간 실행되는 메서드는 인터프리터에 갇혀 워밍업 성능을 저하시킬 수 있다
- 실행 중 Stack Frame을 교체하는 최적화 기법

Inlining

- 함수 호출 오버헤드(Call stack에 stack frame 생성, 인자 전달 및 반환, 복귀 등)를 줄이기 위한 최적화 기법
- 바이트 코드 길이가 작은(35Byte, HotSpot은 325Byte) 함수가 default 값

Code Cache Sweeping

JIT Compiler



Code Cache

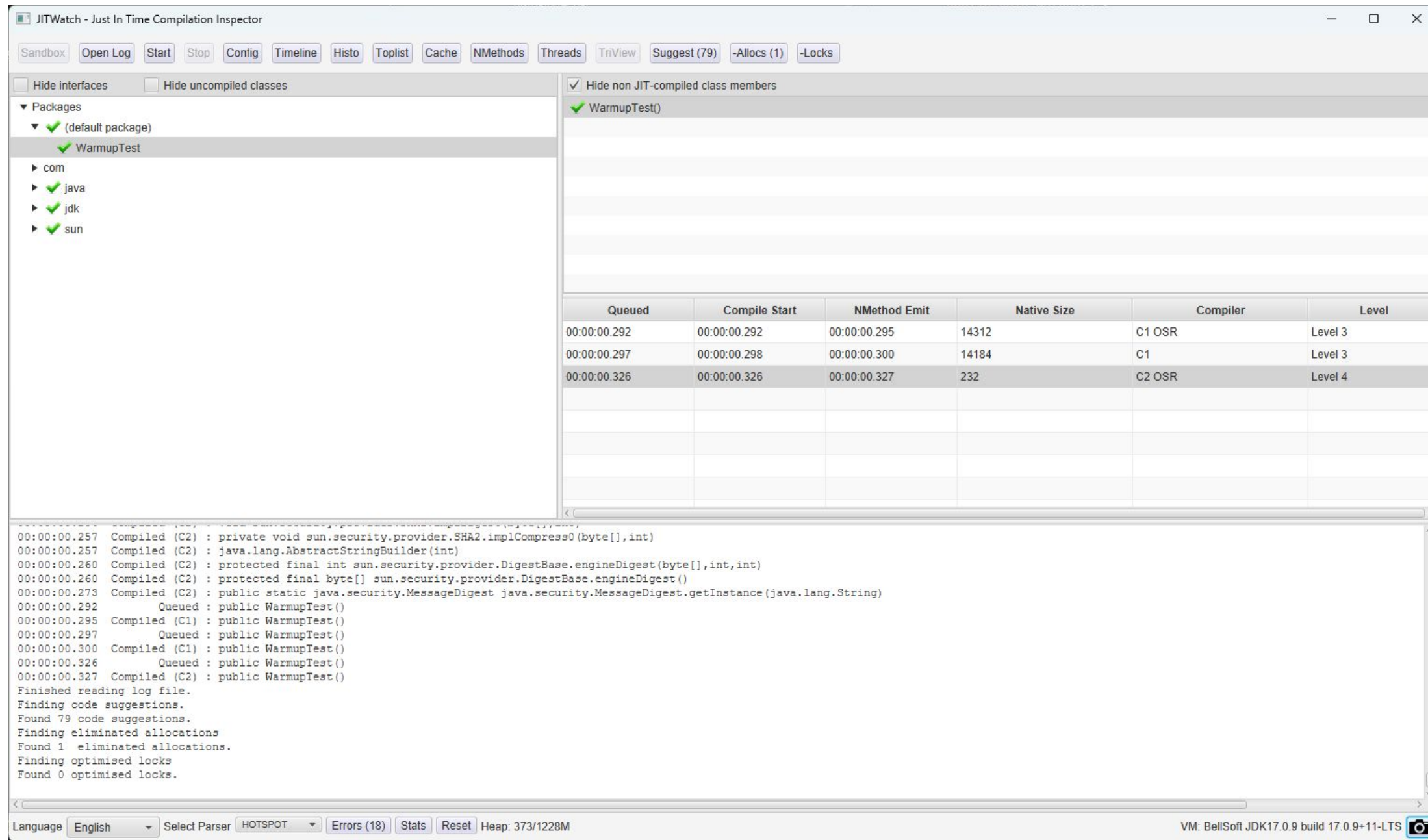
- 컴파일 된 기계어 코드를 저장하는 메모리 공간(non-heap space)
- JVM 성능 튜닝 대상

Code Cache Sweeping

- 코드 캐시가 가득 찼거나 더 이상 필요 없는 코드가 생겼을 때, 해당 공간을 청소하여 메모리를 회수하는 **JVM**의 관리 작업

JITWatch

JIT Compiler



- JIT 로그 분석 및 시각화 도구
- 다양한 테스트 코드 제공
- C1 & C2 Compiler를 이해 & 테스트하기에 매우 적합!!

출처 : <https://github.com/AdoptOpenJDK/jitwatch>

Practice

with JITwatch

QnA